



Estabelecimento de Chave Simétrica e Kerberos

Prof. Dr. Iaçanã Ianiski Weber

Confiabilidade e Segurança de Software

98G08-4

Agradecimentos especiais ao Prof. Avelino Zorzo e aos Autores Christof Paar, Jan Pelzl e Tim Güneysu pelo material.

Índice

- 1 O Problema da Distribuição de Chaves
- 2 Derivação de Chaves (KDF)
- 3 Estabelecimento com um KDC
- 4 Ataques ao KDC Básico
- 5 Needham-Schroeder e Kerberos
- 6 Problemas Remanescentes
- 7 Lições Aprendidas

Conteúdo desta Aula

Nesta aula você vai aprender:

- O **problema da distribuição de chaves** (n^2) e por que ele motiva uma **terceira parte confiável**.
- Como derivar muitas chaves de sessão a partir de um segredo: **funções de derivação de chave (KDF)** e **PBKDF2**.
- **Estabelecimento de chave com um KDC** (*Key Distribution Center*): chaves de longo prazo (**KEK**) vs. chaves de **sessão**.
- Os **ataques** ao protocolo básico: *replay* e confirmação de chave.
- O protocolo **Needham–Schroeder** e o **Kerberos**.
- As **limitações** dos esquemas simétricos (sem PFS, ponto único de falha, escala).

- 1 O Problema da Distribuição de Chaves
- 2 Derivação de Chaves (KDF)
- 3 Estabelecimento com um KDC
- 4 Ataques ao KDC Básico
- 5 Needham-Schroeder e Kerberos
- 6 Problemas Remanescentes
- 7 Lições Aprendidas

Motivação: como duas partes obtêm uma chave secreta?

- Durante quase todo o curso, assumimos que cifras simétricas *já dispõem* de uma chave secreta compartilhada, distribuída por um “canal seguro”.
- Mas **como** essa chave chega às partes? Em uma rede com muitos usuários isso vira um problema sério.

Motivação: como duas partes obtêm uma chave secreta?

- Durante quase todo o curso, assumimos que cifras simétricas *já dispõem* de uma chave secreta compartilhada, distribuída por um “canal seguro”.
- Mas **como** essa chave chega às partes? Em uma rede com muitos usuários isso vira um problema sério.
- Suponha n usuários em que *cada par* precisa de uma chave secreta própria k_{AB} (conhecida só pelos dois). Então:
 - cada usuário guarda $n - 1$ chaves;
 - há $\binom{n}{2} = \frac{n(n-1)}{2} \approx \frac{n^2}{2}$ pares de chaves na rede;
 - ao entrar um novo usuário, é preciso um **canal seguro com cada** um dos demais para instalar as novas chaves.

Isto é o **problema da distribuição de n^2 chaves**: o número de chaves cresce com o *quadrado* do número de usuários.

Exemplo: uma empresa de porte médio

Exemplo 14.1 (Livro)

Uma empresa com **800 funcionários** quer comunicação segura entre todos os pares, com mecanismos simétricos (confidencialidade por cifra; autenticação/integridade por MAC). Então:

- são necessários $\frac{800 \cdot 799}{2} = \mathbf{319\,600}$ pares de chaves simétricas;
 - ao todo, $800 \cdot 799 = 639\,200$ chaves devem ser distribuídas por canais seguros;
 - se o funcionário 801 entra, **801 canais seguros** precisam ser estabelecidos (um para cada um dos 800 já existentes, mais o novo).
-
- Inviável para redes médias/grandes — e impensável para “toda a internet”.
 - **Ideia:** introduzir uma **terceira parte confiável** que reduza o número de chaves de longo prazo de $\approx n^2/2$ para apenas n .

- 1 O Problema da Distribuição de Chaves
- 2 Derivação de Chaves (KDF)
- 3 Estabelecimento com um KDC
- 4 Ataques ao KDC Básico
- 5 Needham-Schroeder e Kerberos
- 6 Problemas Remanescentes
- 7 Lições Aprendidas

Funções de Derivação de Chave (KDF)

Problema: muitas vezes precisamos gerar *várias* chaves a partir de um único segredo (ex.: uma chave-mestra pré-compartilhada), garantindo **frescor** (*freshness*).

- Uma **KDF** computa uma chave derivada k_d a partir de um segredo k e de um valor público s (contador ou *nonce*):

$$k_d = \text{KDF}(k, s)$$

- Para cada s diferente, uma chave k_d diferente $\Rightarrow$ Alice e Bob, partindo de um segredo comum, derivam **múltiplas chaves de sessão** (ex.: com um contador sincronizado).
- Internamente, a KDF usa uma **PRF** (função pseudoaleatória), em geral construída a partir de hash ou cifra de bloco. Exemplo com HMAC:

$$k_d = \text{KDF}(k, s) = \text{HMAC}_k(s)$$

Frescor: se uma chave de sessão for comprometida, o atacante decifra apenas uma quantidade *limitada* de tráfego.

KDF Baseada em Senha: PBKDF2

Uma aplicação central de KDFs: derivar chaves de **senhas** (fáceis de lembrar, mas de baixa entropia). Em vez de guardar a senha, guarda-se a saída da KDF.

- **Mão única:** a partir do arquivo de hashes de senha, o atacante não recupera as senhas — só pode *adivinhar* (força bruta / dicionário / *rainbow tables*).
- **Defesa 1 — *salt*:** valor aleatório e individual usado como s . Garante que senhas iguais gerem hashes diferentes e **anula tabelas pré-computadas**.
- **Defesa 2 — *lentidão*:** *iterar* a PRF (ex.: milhares de vezes) torna a busca por força bruta proporcionalmente mais cara.

$$k_d = \text{PBKDF2}_{\text{PRF}}(\text{Pwd}, s, i, k_{\text{Len}})$$

Pwd = senha; s = *salt*; i = nº de iterações (1024–4096, ou centenas de milhares para segurança de longo prazo); k_{Len} = tamanho da chave. PRF típica: HMAC com SHA-256.

- 1 O Problema da Distribuição de Chaves
- 2 Derivação de Chaves (KDF)
- 3 Estabelecimento com um KDC**
- 4 Ataques ao KDC Básico
- 5 Needham-Schroeder e Kerberos
- 6 Problemas Remanescentes
- 7 Lições Aprendidas

Centro de Distribuição de Chaves (KDC)

- Um **KDC** (*Key Distribution Center*) é um servidor **plenamente confiável** por todos os usuários.
- Ele **compartilha uma chave secreta com cada usuário U** : a **KEK** (*Key Encryption Key*) k_U , pré-distribuída *uma única vez* por canal seguro.
- A KEK é usada para transmitir, de forma segura, as **chaves de sessão** k_{ses} .

KEK k_U (longo prazo)

Não muda. Forma um “canal secreto” permanente entre o KDC e cada usuário.

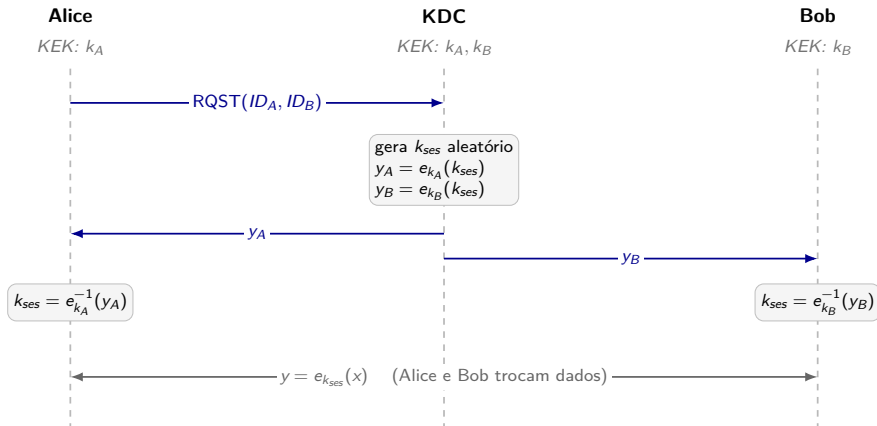
Chave de sessão k_{ses} (efêmera)

Muda a cada sessão (ex.: vida de 1 minuto). Usada para cifrar os dados de fato.

Estes protocolos fazem **transporte de chave** (*key transport*): o KDC *gera* a chave e a entrega — não há negociação.

Protocolo Básico com KDC

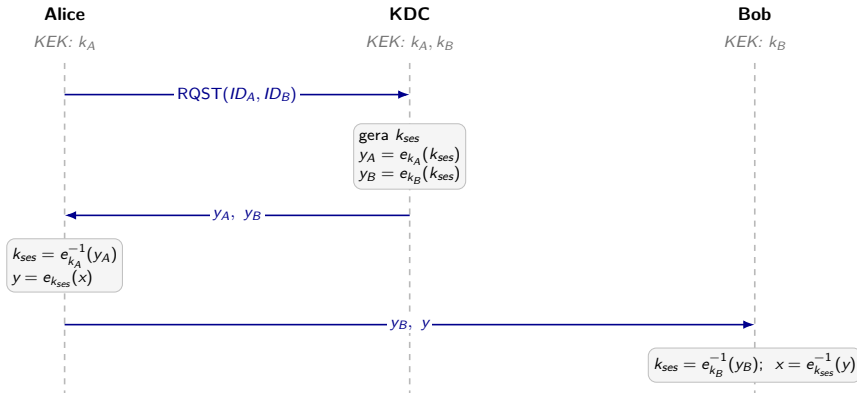
O KDC gera k_{ses} e a envia cifrada sob cada KEK: $y_A = e_{k_A}(k_{ses})$,
 $y_B = e_{k_B}(k_{ses})$.



Pode-se ver as KEKs como “canais secretos” $KDC \leftrightarrow \text{usuário}$; o KDC simplesmente envia k_{ses} por esses canais.

Protocolo com KDC (versão otimizada)

Economiza uma comunicação: o KDC manda *ambos* y_A, y_B para Alice, que repassa y_B a Bob junto com a mensagem.



Vantagem-chave: apenas n **chaves de longo prazo** no sistema ($\text{n\~ao} \approx n^2/2$). Cada usuário guarda só a *sua* KEK; um novo usuário Noah precisa de **um único** canal seguro (com o KDC).

Exemplo Prático: TV por assinatura

- Como as KEKs são de **longo prazo** e as chaves de sessão são **efêmeras**, costuma-se usar cifras diferentes para cada tipo de chave.

Pay-TV

- As KEKs k_U usam **AES** para distribuir as chaves de sessão k_{ses} .
- Cada k_{ses} vive $\approx$ **1 minuto** e cifra o sinal de TV com uma cifra *leve e rápida* (ex.: PRESENT com chave de 80 bits).
- Se uma chave de sessão vazar, o atacante decifra apenas **1 minuto** de conteúdo.

Mas atenção: se uma **KEK** for comprometida, *todo* o tráfego passado e futuro daquele usuário pode ser decifrado. A KEK é o elo crítico.

- 1 O Problema da Distribuição de Chaves
- 2 Derivação de Chaves (KDF)
- 3 Estabelecimento com um KDC
- 4 Ataques ao KDC Básico**
- 5 Needham-Schroeder e Kerberos
- 6 Problemas Remanescentes
- 7 Lições Aprendidas

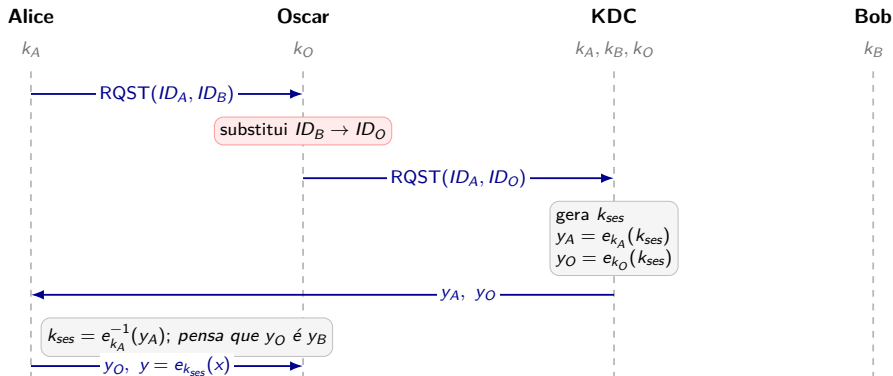
O Protocolo Básico Não Resiste a um Atacante Ativo

Os protocolos anteriores protegem contra um atacante *passivo* (que só escuta), mas **falham** contra um atacante **ativo**, que manipula e injeta mensagens. Dois ataques:

- 1 **Ataque de *replay* (repetição):** nem Alice nem Bob sabem se a chave de sessão recebida é *nova*.
 - Se Oscar obteve uma chave de sessão *antiga* (vazada, ou cuja cifra ficou fraca), ele se passa pelo KDC e **reenvia** os antigos y_A, y_B .
 - Como ele conhece aquela k_{ses} , decifra tudo o que Alice/Bob cifrarem com ela. Falta **frescor de chave**.
- 2 **Ataque de confirmação de chave:** Alice não tem garantia de que a chave recebida é para uma sessão *com Bob* (próximo slide).

Ataque de Confirmação de Chave

Oscar é um usuário *legítimo mas malicioso*. Ele troca a requisição $RQST(ID_A, ID_B)$ por $RQST(ID_A, ID_O)$.



O KDC cria uma sessão $Alice \leftrightarrow Oscar$, mas Alice *acredita* falar com Bob; Oscar intercepta y e decifra. **Causa raiz:** não há **confirmação de chave**.

- 1 O Problema da Distribuição de Chaves
- 2 Derivação de Chaves (KDF)
- 3 Estabelecimento com um KDC
- 4 Ataques ao KDC Básico
- 5 **Needham-Schroeder e Kerberos**
- 6 Problemas Remanescentes
- 7 Lições Aprendidas

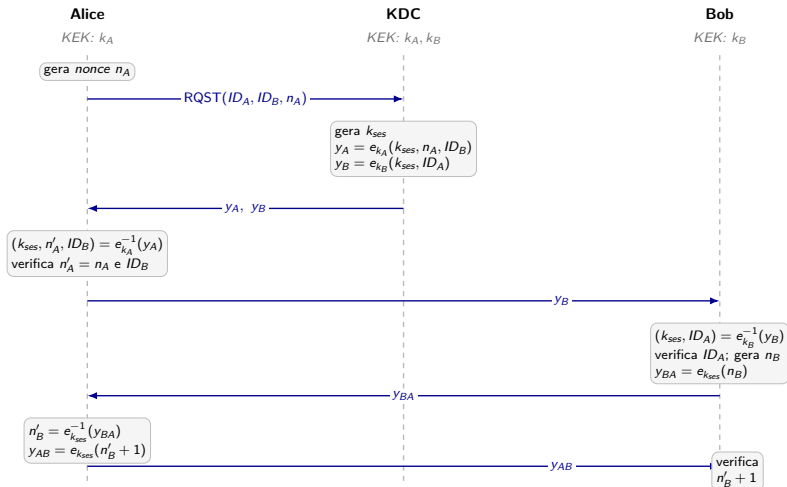
Protocolo Needham–Schroeder

- Esquema mais avançado que protege contra *replay* e confirmação de chave.
- É a **base do Kerberos**, protocolo de autenticação muito usado em redes (RFC, 1993). O KDC é chamado de **servidor de autenticação** (*authentication server*).
- **Ideia central:** introduzir *nonces* (n_A, n_B) — “*numbers used once*” — para garantir **frescor** via **desafio–resposta** (*challenge–response*).

Duas garantias de frescor:

- Alice gera n_A e o recebe de volta cifrado $\Rightarrow$ sabe que k_{ses} foi gerada *após* seu pedido.
- Bob gera n_B , recebe $n_B + 1$ de Alice $\Rightarrow$ confirma que Alice tem a *mesma* k_{ses} .

Needham-Schroeder: o Protocolo



Por que Needham–Schroeder Funciona

- **Frescor / anti-replay:** o *nonce* n_A devolvido dentro de y_A prova que k_{ses} foi gerada *depois* do pedido de Alice (não é chave reutilizada).
- **Autenticação do KDC (desafio–resposta):** Alice desafia o KDC a cifrar n_A sob k_A ; se o n'_A retornado bate, ela sabe que y_A veio mesmo do KDC.
- **Confirmação de chave:** incluir ID_B em y_A garante a Alice que a chave é para uma sessão *com Bob*; incluir ID_A em y_B garante a Bob que a chave é para falar *com Alice*. (Resolve o ataque do slide anterior.)
- **Autenticação mútua:** a troca de n_B e $n_B + 1$ confirma a Bob que Alice de fato possui k_{ses} .

Ainda há um *replay* possível: se Oscar comprometeu uma k_{ses} *antiga*, ele reenvia o antigo y_B a Bob, intercepta y_{BA} e gera um y_{AB} válido. Bob acredita estar em sessão com Alice. (*A vulnerabilidade migra para o comprometimento de chaves de sessão antigas.*)

Kerberos

- Nasceu no **MIT** nos anos **1980**, como protocolo de autenticação em redes.
- É **baseado no Needham-Schroeder**; o KDC é o *servidor de autenticação*.
- A **versão 5** (Steve Miller e Clifford Neuman) foi publicada como **RFC em 1993** e atualizada na **RFC 4120**.
- Originalmente usava **DES**; substituído por **AES em 2005**. **Cuidado** com implementações legadas em DES (quebrável).
- Amplamente usado: **Windows, macOS, Linux**, OpenBSD/FreeBSD.

Escala

Na prática, Kerberos roda com **dezenas de milhares** de usuários. Para aliviar a carga, aumenta-se a *vida* das chaves de sessão. Ainda assim, não escala para “toda a internet”.

- 1 O Problema da Distribuição de Chaves
- 2 Derivação de Chaves (KDF)
- 3 Estabelecimento com um KDC
- 4 Ataques ao KDC Básico
- 5 Needham-Schroeder e Kerberos
- 6 Problemas Remanescentes**
- 7 Lições Aprendidas

Problemas dos Esquemas Baseados em KDC

Mesmo o Needham–Schroeder/Kerberos têm limitações gerais dos esquemas *simétricos*:

- ❶ **Sem *Perfect Forward Secrecy* (PFS).** Se uma KEK k_A vaza, Oscar decifra todo o tráfego *futuro* de Alice e, se guardou mensagens antigas y_A e y , também o tráfego *passado*. Parar de usar a KEK não desfaz o estrago.
- ❷ **Canal seguro na inicialização.** Todo esquema com KDC exige um canal seguro quando um novo usuário entra (para instalar sua KEK).
- ❸ **Requisitos de comunicação.** O KDC precisa ser contatado a cada nova sessão $\Rightarrow$ gargalo em redes com muitos usuários (problema de desempenho).
- ❹ **Ponto único de falha.** A base de dados de KEKs do KDC é crítica: se o **KDC for comprometido**, *todas* as KEKs caem e precisam ser reestabelecidas por canais seguros.

Perfect Forward Secrecy (PFS)

Definição 14.3.1 — Perfect Forward Secrecy

Um protocolo criptográfico possui **Perfect Forward Secrecy (PFS)** se o *comprometimento de chaves de longo prazo não permite* a um atacante obter **chaves de sessão passadas**.

- Nem o Needham–Schroeder nem os protocolos simétricos mais simples oferecem PFS: as chaves de sessão são protegidas *pelas* KEKs de longo prazo; comprometer a KEK expõe as sessões passadas.
- O mecanismo principal para obter PFS é o uso de **técnicas de chave pública** (ex.: Diffie–Hellman efêmero).

Caminho adiante: a criptografia de **chave pública** resolve a distribuição de chaves em escala e habilita PFS — ao custo de exigir **autenticação das chaves públicas** (certificados / PKI).

- 1 O Problema da Distribuição de Chaves
- 2 Derivação de Chaves (KDF)
- 3 Estabelecimento com um KDC
- 4 Ataques ao KDC Básico
- 5 Needham-Schroeder e Kerberos
- 6 Problemas Remanescentes
- 7 Lições Aprendidas

Lições Aprendidas

- Protocolos de **transporte de chave** transferem com segurança uma chave gerada por uma parte; em **acordo de chave**, todas as partes contribuem para o valor final.
- Na maioria dos protocolos simétricos há uma **terceira parte confiável** (KDC); o canal seguro com ela costuma ser necessário *só na inicialização*.
- Esquemas simétricos **não escalam bem** para redes muito grandes e, em geral, **não oferecem PFS**.
- **KDFs** permitem derivar muitas chaves de sessão de uma chave inicial e são amplamente usadas para gerar chaves a partir de **senhas** (ex.: PBKDF2).
- **Kerberos** (baseado em Needham–Schroeder) é o protocolo simétrico de autenticação mais usado na prática.
- Uma **boa gestão de chaves** é crucial e deve ser considerada por *todo o ciclo de vida* do sistema.